

Lineare Algebra - Übungen 4

Klaus Rheinberger, FH Vorarlberg

20. März 2025

i Abgabe in ILIAS

- Alle PYTHON-Aufgaben gesamthaft in einem lauffähigen Jupyter Notebook, siehe Vorlage in ILIAS.
- Alle PAPIER-Aufgaben gesamthaft in einer PDF-Datei inkl. Name auf den gescannten Seiten.

1 Rang (2 + 2 = 4 Punkte)

1. PAPIER: Bestimmen Sie von Hand mittels elementarer Umformungen den Rang von

$$A = \begin{pmatrix} 1 & 0 & -1 & 1 \\ 2 & 3 & 1 & 5 \\ 3 & 2 & -1 & 6 \\ 0 & 5 & 5 & 5 \end{pmatrix}.$$

2. PYTHON: Überprüfen Sie das Ergebnis am Computer.

Lösung: Siehe Papula Bd2 I: Abschnitt 4: Aufgabe 9b. Rang von A ist 3. Siehe Abbildung 1.

```
A = np.array([[1, 0, -1, 1],
              [2, 3, 1, 5],
              [3, 2, -1, 6],
              [0, 5, 5, 5]])
print(f"Rang von A = {np.linalg.matrix_rank(A)}")
```

Rang von A = 3

Endergebnis

Der Rang von A ist 3.

$$\begin{array}{l}
 \left(\begin{array}{cccc|c}
 1 & 0 & -1 & 1 & \\
 2 & 3 & 1 & 5 & \\
 3 & 2 & -1 & 6 & \\
 0 & 5 & 5 & 5 & 1:5
 \end{array} \right) \quad \begin{array}{l} -2 \cdot z_1 \\ -3 \cdot z_1 \end{array} \\
 \hline
 \left(\begin{array}{cccc|c}
 1 & 0 & -1 & 1 & \\
 0 & 3 & 3 & 3 & 1:3 \\
 0 & 2 & 2 & 3 & \\
 0 & 1 & 1 & 1 & -z_2
 \end{array} \right) \\
 \hline
 \left(\begin{array}{cccc|c}
 1 & 0 & -1 & 1 & \\
 0 & 1 & 1 & 1 & \\
 0 & 2 & 2 & 3 & -2 \cdot z_2 \\
 0 & 0 & 0 & 0 &
 \end{array} \right) \\
 \hline
 \left(\begin{array}{cccc|c}
 1 & 0 & -1 & 1 & \\
 0 & 1 & 1 & 1 & \\
 0 & 0 & 0 & 1 & \\
 0 & 0 & 0 & 0 &
 \end{array} \right) \Rightarrow \underline{\underline{\text{Rang}(A)=3.}}
 \end{array}$$

Abbildung 1: Rang

2 Diät (4 Punkte)

Die Formel für die [Cambridge-Diät](#), eine beliebte Diät in den 1980er Jahren, basiert auf jahrelanger Forschung. Um die gewünschten Mengen und Verhältnisse der Nährstoffe zu erreichen, musste man eine Vielfalt von Lebensmitteln einbeziehen. Jedes Lebensmittel liefert mehrere der erforderlichen Inhaltsstoffe, jedoch nicht in den genau richtigen Anteilen. So war zum Beispiel fettfreie Milch eine wichtige Eiweißquelle, enthielt aber zu viel Kalzium. Daher wurde Sojamehl für einen Teil des Eiweißes verwendet, da Sojamehl wenig Kalzium enthält. Allerdings enthält Sojamehl zu viel Fett, so dass Molke hinzugefügt wurde, da sie im Verhältnis zum Kalzium weniger Fett liefert. Leider enthält die Molke zu viele Kohlenhydrate ... Das folgende Beispiel veranschaulicht das Problem in kleinem Maßstab. In der Tabelle Tabelle 1 sind drei der Bestandteile der Diät aufgeführt, zusammen mit den Mengen bestimmter Nährstoffe.

Tabelle 1: Enthaltene Nährstoffmenge (g) pro 100 g der Zutat

Nährstoff	fettfreie Milch	Sojamehl	Molke	Geforderte Nährstoffmenge (g)
Eiweiß	36	51	13	33
Kohlenhydrate	52	34	74	45
Fett	0	7	1.1	3

PYTHON: Wenn möglich, finden Sie eine Kombination aus fettfreier Milch, Sojamehl und Molke, um die geforderten Mengen an Eiweiß, Kohlenhydraten und Fett zu erhalten.

Quelle: Lay, David; Lay, Steven; McDonald, Judi (2021): Linear Algebra and Its Applications. 6th edition, Pearson Education Limited.

Lösung:

```
A = np.array([[36, 51, 13],
              [52, 34, 74],
              [ 0,  7,  1.1]])
b = np.array([[33], [45], [3]])
Ab = np.hstack((A, b))
print(f"Rang(A) = {np.linalg.matrix_rank(A)}")
print(f"Rang(Ab) = {np.linalg.matrix_rank(Ab)}")

# Lösung mit np.linalg.solve:
x = np.linalg.solve(A, b)
print(f"Lösung von Ax=b: x =\n{x}")
```

```
Rang(A) = 3
Rang(Ab) = 3
Lösung von Ax=b: x =
[[0.27722318]
 [0.39192086]
 [0.23323088]]
```

Die Diät erfordert 27.7 g fettfreie Milch, 39.2 g Sojamehl und 23.3 g Molke.

Endergebnis

Die Diät erfordert 27.7 g fettfreie Milch, 39.2 g Sojamehl und 23.3 g Molke.

3 Ill-Conditioned Linear System (2 + 2 = 4 Punkte)

1. PYTHON: Lösen Sie die beiden LGS $Ax = b$

$$0.835x_1 + 0.667x_2 = 0.168$$

$$0.333x_1 + 0.266x_2 = 0.067$$

und

$$0.835x_1 + 0.667x_2 = 0.168$$

$$0.333x_1 + 0.266x_2 = 0.066$$

die sich nur im Wert b_2 unterscheiden.

2. PAPIER: Erklären Sie den großen Unterschied in den Lösungen.

Eine Koeffizientenmatrix wie in diesem Beispiel heißt *ill-conditioned* und das LGS heißt *ill-conditioned system*.

Lösung:

PYTHON:

```

A = np.array([[0.835, 0.667],
              [0.333, 0.266]])
print(f"Rank(A) = {np.linalg.matrix_rank(A)}")
b1 = np.array([[0.168],
              [0.067]])
b2 = np.array([[0.168],
              [0.066]])

x1 = np.linalg.solve(A, b1)
print(f"Lösung des ersten LGS:\n{x1}")
x2 = np.linalg.solve(A, b2)
print(f"Lösung des zweiten LGS:\n{x2}")

```

```

Rank(A) = 2
Lösung des ersten LGS:
[[ 1.]
 [-1.]]
Lösung des zweiten LGS:
[[-665.99999998]
 [ 833.99999998]]

```

PAPIER: Die Koeffizientenmatrix A ist in beiden Fällen die gleiche, und hat den Rang 2. Die Zeilen der Koeffizientenmatrix sind aber fast linear abhängig, d. h. fast parallel. Die Gleichungen, die zu jeder Zeile gehören, entsprechen fast parallelen Geraden, deren Schnittpunkt durch eine kleine Änderung in b stark beeinflusst wird. Die rechten Seiten b_1 und b_2 unterscheiden sich nur um 0.001. Die Lösungen x_1 und x_2 unterscheiden sich jedoch sehr. Das LGS ist *ill-conditioned*, d.h. kleine Änderungen in b führen zu großen Änderungen in x .

Endergebnis

Lösung des ersten LGS: $\begin{pmatrix} 1 \\ -1 \end{pmatrix}$, Lösung des zweiten LGS: $\begin{pmatrix} -665.99999998 \\ 833.99999998 \end{pmatrix}$.

4 Rechnen mit Matrizen (2 + 2 = 4 Punkte)

Gegeben sind folgende zwei Matrizen und ein Vektor:

$$A = \begin{pmatrix} 1 & -2 & 3 \\ 0 & -5 & 4 \\ 4 & -3 & 8 \end{pmatrix}, B = \begin{pmatrix} 1 & 2 \\ 0 & 4 \\ 3 & 7 \end{pmatrix} \text{ und } c = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}$$

1. PAPIER: Berechnen Sie folgende Ausdrücke per Hand.
2. PYTHON: Überprüfen Sie die Ergebnisse mittels Python.

- AB
- BA
- $B^T A$
- $c^T B$
- $c^T (BB^T + A)c$

Hinweis: In Python können Sie das Transponieren einer Matrix A mit $A.T$ durchführen.

Lösung: Siehe Abbildung 2.

$$\begin{aligned}
 AB &= \begin{pmatrix} 1 & -2 & 3 \\ 0 & -5 & 4 \\ 4 & -3 & 8 \end{pmatrix} \begin{pmatrix} 1 & 2 \\ 0 & 4 \\ 3 & 7 \end{pmatrix} = \begin{pmatrix} 1 \cdot 1 + (-2) \cdot 0 + 3 \cdot 3 & 1 \cdot 2 + (-2) \cdot 4 + 3 \cdot 7 \\ 0 \cdot 1 + (-5) \cdot 0 + 4 \cdot 3 & 0 \cdot 2 + (-5) \cdot 4 + 4 \cdot 7 \\ 4 \cdot 1 + (-3) \cdot 0 + 8 \cdot 3 & 4 \cdot 2 + (-3) \cdot 4 + 8 \cdot 7 \end{pmatrix} = \\
 &= \begin{pmatrix} 10 & 15 \\ 12 & 8 \\ 28 & 52 \end{pmatrix}. \quad B \cdot A \text{ ist Multiplikation von} \\
 &\quad 3 \times 2 \text{ und } 3 \times 3 \Rightarrow \text{nicht wohldefiniert.} \\
 B^T A &= \begin{pmatrix} 1 & 0 & 3 \\ 2 & 4 & 7 \end{pmatrix} \begin{pmatrix} 1 & -2 & 3 \\ 0 & -5 & 4 \\ 4 & -3 & 8 \end{pmatrix} = \begin{pmatrix} 1 \cdot 1 + 0 \cdot 0 + 3 \cdot 4 & 1 \cdot (-2) + 0 \cdot (-5) + 3 \cdot (-3) & 1 \cdot 3 + 0 \cdot 4 + 3 \cdot 8 \\ 2 \cdot 1 + 4 \cdot 0 + 7 \cdot 4 & 2 \cdot (-2) + 4 \cdot (-5) + 7 \cdot (-3) & 2 \cdot 3 + 4 \cdot 4 + 7 \cdot 8 \end{pmatrix} = \\
 &= \begin{pmatrix} 13 & -11 & 27 \\ 30 & -45 & 78 \end{pmatrix}. \quad c^T B = (1 \ 2 \ 3) \begin{pmatrix} 1 & 2 \\ 0 & 4 \\ 3 & 7 \end{pmatrix} = (1 \cdot 1 + 2 \cdot 0 + 3 \cdot 3, 1 \cdot 2 + 2 \cdot 4 + 3 \cdot 7) = \\
 &\quad = (10 \ 31). \\
 c^T (B B^T + A) c &= (c^T B B^T + c^T A) c = c^T B B^T c + c^T A c = \\
 &= c^T B \cdot (c^T B)^T + c^T A c = \\
 &= (10 \ 31) \begin{pmatrix} 10 \\ 31 \end{pmatrix} + (1 \ 2 \ 3) \begin{pmatrix} 1 & -2 & 3 \\ 0 & -5 & 4 \\ 4 & -3 & 8 \end{pmatrix} \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix} = \\
 &= 100 + 961 + (1 \ 2 \ 3) \begin{pmatrix} 1-4+9 \\ 0-10+12 \\ 4-6+24 \end{pmatrix} = 1061 + (1 \ 2 \ 3) \begin{pmatrix} 6 \\ 2 \\ 22 \end{pmatrix} = \\
 &= 1061 + 6 + 4 + 66 = \underline{1137}.
 \end{aligned}$$

Abbildung 2: Rechnen mit Matrizen

```

A = np.array([[1,-2,3],
              [0,-5,4],
              [4,-3,8]])
B = np.array([[1, 2],
              [0, 4],
              [3, 7]])
c = np.array([[1, 2, 3]]).T

print(f"AB = \n{A@B}")
print('BA ist nicht wohldefiniert!')
print(f"B^T A = \n{B.T@A}")
print(f"c^T B = \n{c.T@B}")
print(f"c^T (B B^T + A) c = \n{c.T@(B@B.T + A)@c}")

```

AB =

```

[[10 15]
 [12  8]
 [28 52]]
BA ist nicht wohldefiniert!
B^T A =
[[ 13 -11  27]
 [ 30 -45  78]]
c^T B =
[[10 31]]
c^T(BB^T + A)c =
[[1137]]

```

Endergebnis

$$AB = \begin{pmatrix} 10 & 15 \\ 12 & 8 \\ 28 & 52 \end{pmatrix}, BA \text{ ist nicht wohldefiniert}, B^T A = \begin{pmatrix} 13 & -11 & 27 \\ 30 & -45 & 78 \end{pmatrix}, c^T B = (10 \ 31),$$

$$c^T(BB^T + A)c = 1137$$

5 Adjazenz-Matrix (2 + 2 = 4 Punkte)

Eine Fluglinie fliegt fünf verschiedene Stationen an. Die möglichen Verbindungen sind in Abbildung 3 dargestellt.

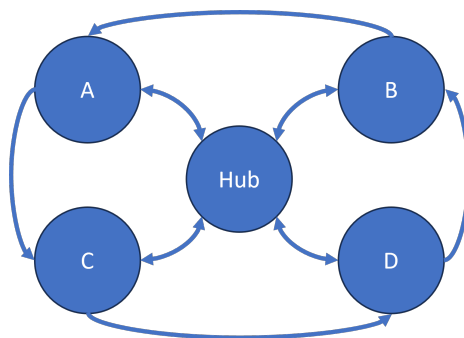


Abbildung 3: Flugplan

1. PYTHON: Erstellen Sie die Adjazenz-Matrix C für die Fluglinie. Dabei gilt

$$C_{ij} = \begin{cases} 1 & \text{wenn es einen Direktflug von } i \text{ nach } j \text{ gibt} \\ 0 & \text{sonst} \end{cases}$$

Um zu bestimmen, wie viele mögliche Flugrouten es zwischen 2 Stationen gibt, kann die Matrixpotenz, also $C^k = C \cdot C \cdot \dots$ verwendet werden, siehe z. B. [Wikipedia](#). Das heißt z. B., die Anzahl der möglichen Flugrouten mit *genau einem Umstieg* zwischen zwei beliebigen Stationen sind die Einträge in der Matrix C^2 . Und die Anzahl der möglichen Flugrouten mit *genau zwei Umstiegen* sind die Einträge in der Matrix C^3 .

2. PYTHON: Bestimmen Sie die Anzahl der Flugrouten zwischen zwei beliebigen Stationen mit (a) genau einem Umstieg und (b) mit genau zwei Umstiegen. (c) Wie viele Routen gibt es vom Startort A zum Zielort B mit *maximal* zwei Umstiegen?

Lösung:

```
# Adjazenz-Matrix:
# Zeilen entsprechen dabei dem Abflugsort und die Spalten dem Zielort
# Reihenfolge der Stationen sind hier alphabetisch sortiert, also: A, B, C, D, Hub
C = np.array([[0,0,1,0,1],
              [1,0,0,0,1],
              [0,0,0,1,1],
              [0,1,0,0,1],
              [1,1,1,1,0]])
print(f"Adjazenzmatrix C = \n{C}")
print(f"Anzahl aller Routen mit genau einem Umstieg = C^2 = \n{C@C}")
print(f"Anzahl aller Routen mit genau zwei Umstiegen = C^3 = \n{C@C@C}")

# Die Summe ergibt die Anzahl an Verbindungen mit maximal 2 Umstiegen
C_all = C + C@C + C@C@C
print(f"Matrix der Anzahl an Routen mit maximal 2 Umstiegen = \n{C_all}")
print(f"Anzahl an Verbindungen zw. A und B mit maximal 2 Umstiegen: {C_all[0,1]}")
```

```
Adjazenzmatrix C =
[[0 0 1 0 1]
 [1 0 0 0 1]
 [0 0 0 1 1]
 [0 1 0 0 1]
 [1 1 1 1 0]]
Anzahl aller Routen mit genau einem Umstieg = C^2 =
[[1 1 1 2 1]
 [1 1 2 1 1]
 [1 2 1 1 1]
 [2 1 1 1 1]
 [1 1 1 1 4]]
Anzahl aller Routen mit genau zwei Umstiegen = C^3 =
[[2 3 2 2 5]
 [2 2 2 3 5]
 [3 2 2 2 5]
 [2 2 3 2 5]
 [5 5 5 5 4]]
Matrix der Anzahl an Routen mit maximal 2 Umstiegen =
[[3 4 4 4 7]
 [4 3 4 4 7]
 [4 4 3 4 7]
 [4 4 4 3 7]
 [7 7 7 7 8]]
Anzahl an Verbindungen zw. A und B mit maximal 2 Umstiegen: 4
```

Endergebnis

(a) C^2 , (b) C^3 , (c) entsprechendes Element aus $C + C^2 + C^3$